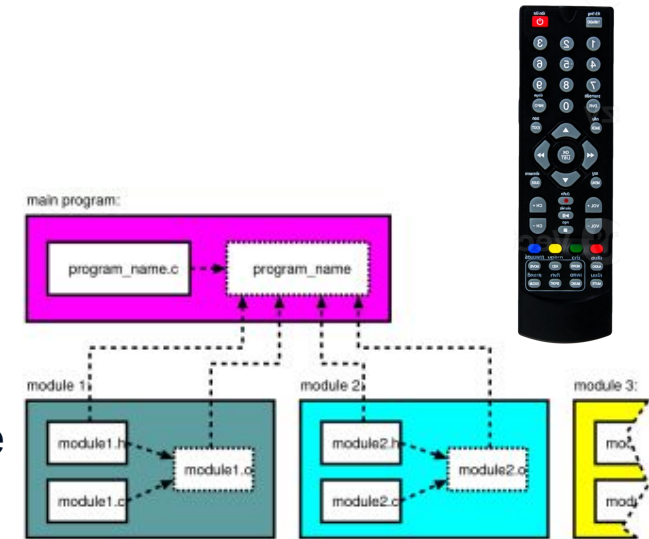




Functional Programming and Modularity

Functional Programming and Modularity

- ❖ **Functional programming** is a programming paradigm where programs are constructed by applying and composing functions.
 - Functions are the main building blocks of a program.
- ❖ A module is a self-contained part of a system that has a **well-defined interface** and an **implementation** to hide the details in order to provide some functionalities for the system
- ❖ Modularity helps us to
 - Organize large programs in smaller parts (modules)
 - Make testable, understandable and reusable code
 - Provide clear and flexible interfaces
 - Have separate compilation and make changes easier
 - Hide implementation and support operations on data structure
 - Create abstract data type and hide details



Functional Programming and Modularity

- ❖ C++ supports modularity using both functional and object oriented programming.
- ❖ Generally there are two types of module
 - Single instance modules which have a single set of data to manage
 - Multiple instance modules which shall be able to manage multiple sets of data
- ❖ The smallest possible module consists of a **header file** and a **source file**.
 - The **header file** contains the module's public interface.
 - According to encapsulation principles it shall only reveal what which is required by the module clients. Even if a module does not manage any data it shall follow this principle. Generally encapsulation protects objects from unwanted access by clients.
 - There are cases in which a module is can/shall be implemented in only a header file.
 - The **source file** contains the implementation of the public interface in order to hide the details of the module and restrict access to identifiers out of the source file.

Modularity

- ❖ To make a module we shall follow some rules and conventions
 - Choose a unique name for the module
 - Make a header file and a source file with the same name as the module name.
 - All the accessible identifiers in the header file shall start with name of the module
 - It also is possible to use namespaces
 - Always we shall have an inclusion guard for a header file.
 - Declare functionalities and all needs of client of the module in the header file.
 - The header file and identifiers in the header file shall be well-described.
 - The header file is the public interface and descriptions are like the module manual.
 - Always include the header file in the source file.
 - Ensure that clients of the module have access only to those identifiers which are required to use the functionalities provided by the module; not more, not less.

Modularity

❖ A C Module Example

```
#ifndef MODULE_H
#define MODULE_H

#ifdef __cplusplus
extern "C"
{
    /* ===== */
    /* ===== include files ===== */
    /* ===== */

    /* Inclusion of system and local header files goes here */

    /* ===== */
    /* ===== constants ===== */
    /* ===== */

    /* #define and enum statements go here */

    /* ===== */
    /* ===== public data ===== */
    /* ===== */

    /* Definition of public (external) data types go here */

    /* ===== */
    /* ===== public functions ===== */
    /* ===== */

    /* Function prototypes for public (external) functions go here */

#ifdef __cplusplus
}
#endif
#endif /* MODULE_H */
```

```
/* ===== */
/* ===== include files ===== */
/* ===== */

/* Inclusion of system and local header files goes here */

/* ===== */
/* ===== constants ===== */
/* ===== */

/* #define and enum statements go here */

/* ===== */
/* ===== global variables ===== */
/* ===== */

/* Global variables definitions go here */

/* ===== */
/* ===== private data ===== */
/* ===== */

/* Definition of private datatypes go here */

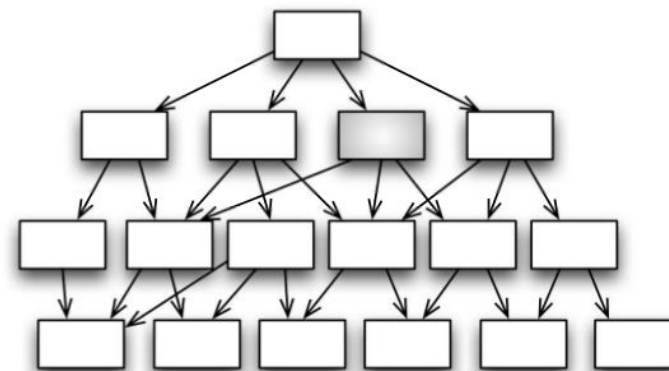
/* ===== */
/* ===== private functions ===== */
/* ===== */

/* Function prototypes for private (static) functions go here */

/* ===== */
/* ===== All functions by section ===== */
/* ===== */

/* Functions definitions go here, organised into sections */
```

- **Single Responsibility Principle**
- **Open Closed Principle**
- **Liskov Substitution Principle**
- **Interface Segregation Principle**
- **Dependency Inversion Principle**



- A module should have a single responsibility
- It should do one thing and do it well
- It should have a single reason to change
- It leads to modules with good cohesion and understandability

Modularity & SOLID Design

❖ Open Closed Principle (OCP)

- A module should be open for extension but closed for modification
- It helps to make software more maintainable and adaptable over time.
- Adding new behavior or functionality to a module without changing its existing code.
- Once a module is written and tested, it should not be changed. Avoid modifying existing code, especially if it's already in use elsewhere.

It prevents introducing bugs into tested and working code and makes your code backward compatible.

❖ Liskov Substitution Principle (LSP)

- A client modules should not care which kind of server module it is working with
- Modules with the same interface should be substitutable without any special knowledge in the calling code
- E.g. the **client** code should not behave differently when interacting with the **server's test double**

Modularity & SOLID Design

❖ Interface Segregation Principle (ISP)

- The client modules should not depend on fat interfaces
- Interfaces should be tailored and very focused to the client's needs
- It limits dependencies, makes code more easily tested and ported

❖ Dependency Inversion Principle (DIP)

- High-level modules shouldn't depend on low-level modules
- Dependencies should be broken with abstractions and interfaces
 - We hide the implementation details behind an interface
 - A client calls a server through a function pointer
 - A server calls a client back through a function pointer
 - An ADT hides the details of a data type

C++ and Functional Programming

- ❖ Only one definition of any variable, function, class type, enumeration type, or template is allowed in any one translation unit. Definitions are declarations that fully define the entity introduced by the declaration. There are exceptions. Look at [One Definition Rule](#)
- ❖ If in the scope of a **source file** we have a:
 - A **static** variable or function, its scope is limited to the file and out of the file is not accessible.
 - Note that the default storage class of a function is **extern** and globally it is accessible.
 - A type or macro, its scope is limited to the file.
 - A **const** variable, its scope by default is the file.
 - But it is possible to make it **extern** and using **extern** get access to it out of the file.
 - A **constexpr** variable, its scope is limited to the file.
 - Impossible to get access to it out of the file.
 - An **inline** function definition must be reachable. An **inline** function may be defined in a header

C++ and Functional Programming

- ❖ If an identifier in another header is used only in the source file of the module, the header file shall be included only in the source file. In this way we can make a module private in another module. E.g. only implementation of module A depends on module B; header file of B shall be included only in the source file of A.
- ❖ Generally we shall not have program scope global variables and therefore we shall not define non-constexpr object in a header file.
- ❖ `constexpr` functions are functions that may be evaluated at compile-time. E.g. `constexpr int add(int x, int y) { return (x + y); }`
 - `constexpr` functions are implicitly `inline` functions
 - `constexpr` functions can also be evaluated at runtime
- ❖ In C++ 17 it is also possible to have `inline` variables

```
#include <iostream>

constexpr int add(int x, int y) { return (x + y); }

int main(void) {
    constexpr double gravity{9.8}; // Compile-time const
    int id;
    std::cout << "Enter your age: ";
    std::cin >> id;
    const int my_id{id}; // A runtime constant
    constexpr int a{add(20, 30)};

    // Error: value of id is unknown during compilation
    constexpr int your_id{add(10, id)};
    return 0;
}
```

Modularity

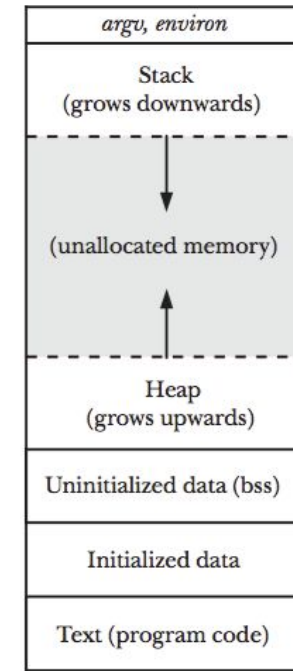
- ❖ In a **single instance** module we can define variables and function as static to make them them private. If it is required, make the private data accessible through the module's public interface. By defining a set of function prototypes in the header file.
- ❖ In **multiple instance** module to hide the data details we shall create incomplete types to abstract data. To create an incomplete type we shall:
 - Have a forward declaration of the type in the header file.
 - E.g. In C++ `struct person`; or in C `typedef struct person person`;
 - Define the type in the source file. E.g. `struct person { int id; int age; }`
 - We can not create instances of `person` out of the source file.
 - We can only create `person` pointers and an incomplete pointer type cannot be dereferenced.
 - To create instances during run time we shall use dynamic memory allocation

Modularity

- ❖ When we allocate memory dynamically we are responsible to release it.
- ❖ Usually in a **single instance** module we have an initialization and a deinitialization functions
- ❖ In **multiple instance** module we shall have a create and and a destroy functions
 - The create function is used to create instances using dynamic memory allocation
 - The destroy function is used to release the dynamically allocated memory for an instance
- ❖ **Dynamic Memory Management**
 - In this mechanism we can allocate and deallocate memory dynamically during runtime
 - The heap or free store is a large pool of memory; it can be used for storing dynamic size data
 - The programmer is responsible to manage the dynamic memory
 - Data stored in the heap, is alive until we release the allocated memory for the data
 - Unlike local variables which exist only in their scopes

Dynamic Memory Management

- ❖ What we know about different segments of a program
 - **Code Segment** is used to store the program code.
 - The global and static variables and constant string literals are stored in the **Data Segment**.
 - A **Stack** is used to store local variables in functions and
 - and the required data for functions calls and returns
 - Heap is used to for dynamic objects during run time
- ❖ Sometimes size of data a program should process is unknown during compilation and it will be specified during run time. E.g. When we ask a user to enter her name
 - Size of the name is unknown during compilation



Memory Layout of a C Program

Environment is the used for the environment variables and arguments passed to the program. **BSS** (Block Started by Symbol) includes the uninitialized global and static variables. **DS** includes the global initialized constants , static variables and string literals.

Dynamic Memory Management

- ❖ In C/C++ dynamic memory management can be done in two ways
 - The C functions defined in **stdlib**:
 - `void *malloc(size_t size);`
 - `void *calloc(size_t count, size_t size);`
 - `void *realloc(void *ptr, size_t size);`
 - `void free(void *ptr);`
 - The **new** and **delete** operators of C++
 - We shall not mix them. E.g. using **malloc** to allocate memory and free it using **delete**
 - ***As the best practice; use only new and delete***
- ❖ ***When we allocate memory always we have to check for failure***
- ❖ There are some useful functions in **string.h/cstring** to operate on blocks of memory
 - E.g. `memset`, `memcmp`, `memcpy`, `memmove` and etc.

Functional Programming and Modularity

❖ Some useful links

- [SOLID Agile Development](#)
- [SOLID Principles made easy](#)
- [SOLID Principle in Programming](#)
- [SOLID Principles: Explanation and examples](#)
- [C++ Reference](#)
- [Standard C++ Library Reference](#)
- [C++ Tutorial](#)
- [C++ Language](#)
- [C++ Tutorial](#)
- [C++ Full Course For Beginners](#)
- [C++ Tutorial for Beginners - Full Course](#)
- [C++ Tutorial 2021](#)
- [C++ Programming Tutorials Playlist](#)
- [Linked List Data Structure](#)
- [Stack Data Structure](#)
- [Queue Data Structure](#)
- [Circular Buffer Structure](#)
- [Ring Buffer \(Circular Buffer\)](#)
- [Binary Tree Data Structure](#)
- [Binary Search Tree \(BST\)](#)
- [Data Structures Easy to Advanced Course](#)